

Lab Sheet: ArrayList and Collections in Java

Course: OOP Lab / CSE 1116

Topic: Dynamic Arrays, Basic Operations, Sorting with Comparable & Comparator, Filtering

Level: Beginner to Intermediate

Duration: 3 hours

Prepared by: Ashraful Islam Paran, Dept. of CSE, UIU

1. Objectives

By the end of this lab, students will be able to:

- Understand how to declare and use dynamic arrays using `ArrayList`.
- Perform basic operations: add, get, set, remove, and search elements.
- Create an `ArrayList` of custom Class objects.
- Sort lists naturally using `Comparable`.
- Define custom sorting logic using `Comparator`.
- Filter elements based on specific criteria.

2. Introduction to ArrayList

The `ArrayList` class supports dynamic arrays that can grow as needed, unlike standard fixed-size arrays. An `ArrayList` is a variable-length array of object references and can contain duplicate elements.

When the initial size of an `ArrayList` is exceeded, it dynamically allocates larger memory on the heap, copies the current elements to the new memory, adds the new item, and deletes the old memory. The `ArrayList` inherits the `AbstractList` class and implements the `List` interface.

3. Basic ArrayList Operations

```
import java.util.ArrayList;
import java.util.Collections;

public class ArrayListDemo {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>(); // Declaring an
            ↪ ArrayList

        // Adding elements
        list.add("String 1");
        list.add("String 2");
        list.add("String 3");
        list.add(1, "String 4"); // Add to specific index

        // Accessing and Changing elements
        String s1 = list.get(0);
        list.set(1, "String 5"); // Change item at specific index

        // Removing and Searching
        list.remove(2); // Remove specific item of an index
        boolean hasElement = list.contains("String 3");
        int index = list.indexOf("String 1"); // Returns -1 if not present

        // Sorting in ascending and descending order
        Collections.sort(list);
        Collections.sort(list, Collections.reverseOrder());

        System.out.println("Size: " + list.size()); // Size of arraylist
        list.clear(); // Clear the list
    }
}
```

4. ArrayList of Class Objects and Filtering

We can create an ArrayList of custom objects (e.g., Student). We can also iterate through the list and filter it based on specific criteria.

```
import java.util.ArrayList;

class Student {
    int id;
    String name;
    double cgpa;

    Student(int id, String name, double cgpa) {
        this.id = id;
        this.name = name;
        this.cgpa = cgpa;
    }

    @Override
    public String toString() {
        return name + "(" + id + ", " + cgpa + ")";
    }
}

public class FilterDemo {
    public static void main(String[] args) {
        ArrayList<Student> list = new ArrayList<>();
        list.add(new Student(1, "Sakib", 3.99));
        list.add(new Student(4, "Fatema", 2.5));
        list.add(new Student(5, "Ayesha", 2.00));

        // Filter using a criteria (CGPA > 3)
        ArrayList<Student> filteredList = new ArrayList<>();
        for(Student s : list) {
            if(s.cgpa > 3) {
                filteredList.add(s);
            }
        }
        System.out.println(filteredList);
    }
}
```

5. Sorting Objects using Comparable

To sort objects naturally, a class can implement the Comparable interface and override the compareTo method.

```
import java.util.ArrayList;
import java.util.Collections;

class Student implements Comparable<Student> {
    int id;
    String name;
    double cgpa;
    // ... Constructor omitted for brevity ...

    @Override
    public int compareTo(Student o2) {
        // Sort using cgpa, if same sort alphabetically using name
        if(this.cgpa == o2.cgpa) {
            return this.name.compareTo(o2.name);
        }
        return this.cgpa > o2.cgpa ? 1 : -1;
    }
}

public class ComparableDemo {
    public static void main(String[] args) {
        ArrayList<Student> list = new ArrayList<>();
        // ... Add students ...
        Collections.sort(list); // Sorts based on compareTo logic
    }
}
```

6. Sorting Objects using Comparator

Comparator is useful when you need multiple sorting logics (e.g., sort by ID, then separately by CGPA) without modifying the original class. This can be done via separate classes or anonymous inner classes.

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;

public class ComparatorDemo {
    public static void main(String[] args) {
        ArrayList<Student> list = new ArrayList<>();
        list.add(new Student(1, "Sakib", 3.99));
        list.add(new Student(2, "Rahim", 3.75));

        // Sort Alphabetically using name via Anonymous Inner Class
        Collections.sort(list, new Comparator<Student>() {
            @Override
            public int compare(Student o1, Student o2) {
                return o1.name.compareTo(o2.name);
            }
        });

        // Sort using id
        Collections.sort(list, new Comparator<Student>() {
            @Override
            public int compare(Student o1, Student o2) {
                return o1.id - o2.id; // Ascending order
            }
        });

        // Sort using cgpa (descending order)
        Collections.sort(list, new Comparator<Student>() {
            @Override
            public int compare(Student o1, Student o2) {
                if(o2.cgpa > o1.cgpa) return 1;
                return -1;
            }
        });
    }
}
```

Tasks

Task 1 – Basic List Operations (20 min)

- Create an ArrayList of Integers. Add numbers from 10 to 50.
- Check if the number 30 is in the list. If it is, find its index and remove it.
- Print the list in descending order.

Task 2 – Employee Management System (25 min)

- a. Create an `Employee` class with `id`, `name`, and `salary`.
- b. Create an `ArrayList` of 5 employees.
- c. Filter and print a new list containing only employees earning more than 50,000 BDT.

Task 3 – Multi-level Sorting (20 min)

- a. Using the `Employee` class, implement the `Comparable` interface.
- b. Sort the employees by `salary` in descending order. If salaries are identical, sort them alphabetically by name.

Task 4 – Dynamic Inventory Sorter (25 min)

You are building an inventory system for a store.

- a. Create a `Product` class (`productId`, `productName`, `price`, `stockQty`).
- b. Use a `Scanner` to add products to an `ArrayList` dynamically until the user types "STOP".
- c. Create a menu allowing the user to view the list sorted by:
 - Price (Lowest to Highest) using an anonymous `Comparator`.
 - Stock Quantity (Highest to Lowest) using a separate `Comparator` class.

Task 5 – Merging and Filtering Lists (25 min)

You have two different branches of a library, each maintaining an `ArrayList` of `Book` objects (`isbn`, `title`, `rating` out of 5.0).

- a. Create `list1` and `list2` with dummy book data.
- b. Merge `list2` into `list1`.
- c. Using the merged list, create a `TopRatedBooks` list containing only books with a rating of 4.5 or higher.
- d. Display the `TopRatedBooks` list sorted alphabetically by title.

Keep Coding! Mastering `ArrayLists` and `Collections` is crucial for managing dynamic data and writing efficient algorithms in Java.